



PHASE 2 | TASK 2

Stream into Bronze and Silver

Kafka events landed raw in Bronze, then parsed, typed and de-duplicated into Silver with Structured Streaming



STEP 1

Store the Kafka Credentials as Secrets

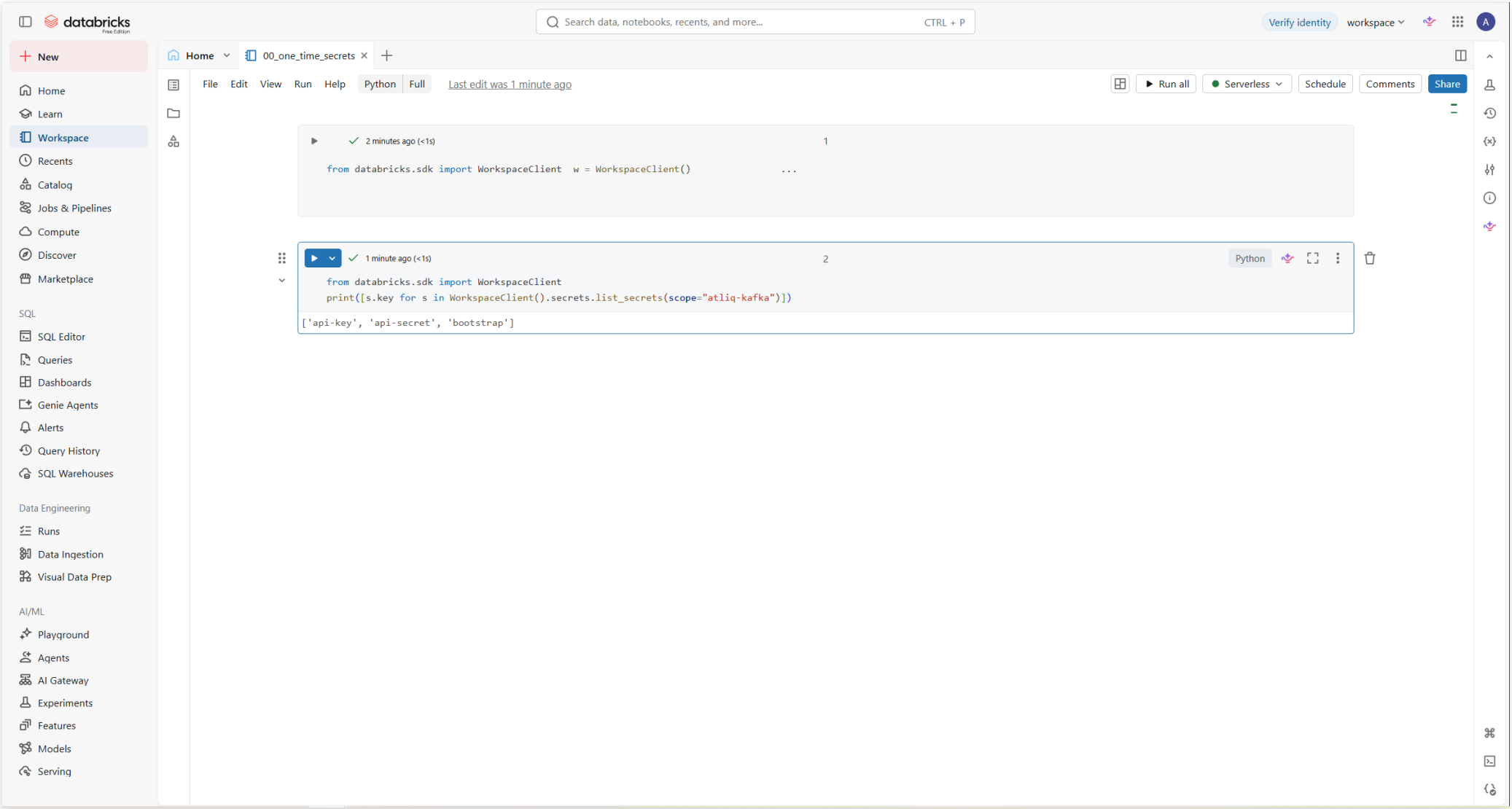
WHAT

Created the Databricks secret scope `atliq-kafka` holding the bootstrap server, API key and API secret, using a one-time scratch notebook that was deleted straight after. The pipeline notebook reads them with `dbutils.secrets.get`.

WHY

The starter notebook expects credentials pasted into its config cell, but that notebook is committed to a public repo. A secret scope keeps the values out of code, Git and screenshots; only the key names are ever displayed.

STEP 1 | Store the Kafka Credentials as Secrets



Secret scope atliq-kafka lists api-key, api-secret and bootstrap. Values are never returned.

2

STEP 2

Land Raw Events in Bronze

WHAT

Read the topic with `spark.readStream.format("kafka")` over SASL_SSL and appended the raw records to `atliq.streaming.bronze_order_events`: key and value cast to string, plus topic, partition, offset, Kafka timestamp and an `ingested_at` audit column.

WHY

Bronze keeps everything exactly as received, the same rule as Phase 1, so any later logic can be replayed from it. The first run read from the earliest offset and landed 879 rows, exactly the 879 events the producer sent in Task 1.

STEP 2 | Land Raw Events in Bronze (1 of 2)

```
41 - # TODO: Task 1 - your Bronze stream here
42
54
55 + # Confluent login. On Databricks, Kafka classes are shaded, hence the kafkashaded prefix.
56 + JAAS = (
57 +     "kafkashaded.org.apache.kafka.common.security.plain.PlainLoginModule required "
58 +     f'username="{KAFKA_API_KEY}" password="{KAFKA_API_SECRET}";'
59 + )
60 +
61 + bronze_df = (
62 +     spark.readStream.format("kafka")
63 +     .option("kafka.bootstrap.servers", KAFKA_BOOTSTRAP)
64 +     .option("subscribe", TOPIC)
65 +     .option("startingOffsets", "earliest") # first run only; afterwards the checkpoint decides
66 +     .option("kafka.security.protocol", "SASL_SSL")
67 +     .option("kafka.sasl.mechanism", "PLAIN")
68 +     .option("kafka.sasl.jaas.config", JAAS)
69 +     .load()
70 +     # Bronze keeps everything, no parsing: binary key/value become strings
71 +     .select(
72 +         F.col("key").cast("string").alias("key"),
73 +         F.col("value").cast("string").alias("value"),
74 +         "topic", "partition", "offset",
75 +         F.col("timestamp").alias("kafka_ts"),
76 +         F.current_timestamp().alias("ingested_at"),
77 +     )
78 + )
79 +
80 + bronze_q = (
81 +     bronze_df.writeStream
82 +     .outputMode("append")
83 +     .option("checkpointLocation", f"{CKPT}/bronze") # Bronze's own checkpoint
84 +     .trigger(availableNow=True) # serverless: process all new events, then stop
85 +     .toTable(BRONZE)
86 + )
87 + bronze_q.awaitTermination() # Silver reads this table, so let Bronze finish first
88 + print("Bronze rows:", spark.table(BRONZE).count())
89 +
90 +
```

Commit 29d3134: the Bronze TODO replaced by the Kafka readStream and a Delta append with its own checkpoint.

STEP 2 | Land Raw Events in Bronze (2 of 2)

The screenshot displays the Databricks interface with a Python notebook titled "01_kafka_stream_processing". The notebook code is as follows:

```
# Confluent login. On Databricks, Kafka classes are shaded, hence the kafkashaded prefix.
JAAS = (
    "kafkashaded.org.apache.kafka.common.security.plain.PlainLoginModule required "
    f'username="{KAFKA_API_KEY}" password="{KAFKA_API_SECRET}";'
)

bronze_df = (
    spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", KAFKA_BOOTSTRAP)
    .option("subscribe", TOPIC)
    .option("startingOffsets", "earliest") # first run only; afterwards the checkpoint decides
    .option("kafka.security.protocol", "SASL_SSL")
    .option("kafka.sasl.mechanism", "PLAIN")
    .option("kafka.sasl.jaas.config", JAAS)
    .load()
    # Bronze keeps everything, no parsing: binary key/value become strings
    .select(
        F.col("key").cast("string").alias("key"),
        F.col("value").cast("string").alias("value"),
        "topic", "partition", "offset",
        F.col("timestamp").alias("kafka_ts"),
        F.current_timestamp().alias("ingested_at"),
    )
)

bronze_q = (
    bronze_df.writeStream
    .outputMode("append")
    .option("checkpointLocation", f"{CKPT}/bronze") # Bronze's own checkpoint
    .trigger(availableNow=True) # serverless: process all new events, then stop
    .toTable(BRONZE)
)

bronze_q.awaitTermination() # Silver reads this table, so let Bronze finish first
print("Bronze rows:", spark.table(BRONZE).count())
```

The output of the notebook shows the schema of the data written to the Bronze table:

```
key: string
value: string
topic: string
partition: integer
offset: long
kafka_ts: timestamp
ingested_at: timestamp
```

Below the schema, the output indicates: "Bronze rows: 879".

First run: Bronze rows 879, matching the producer's 879 events sent, 0 undelivered.



STEP 3

Inspect the Bronze Records

WHAT

Queried the latest Bronze rows: Kafka key, partition, offset and timestamp sit next to the raw JSON and the time Databricks landed each record.

WHY

kafka_ts shows the events were produced at 07:55 UTC and ingested_at shows they were read hours later. Kafka's 7-day retention held them until Databricks was ready, which is the decoupling streaming relies on: producer and consumer never need to run at the same time.

STEP 3 | Inspect the Bronze Records

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Home

Tree view: ON

ashish@alp...

01_kafka_stream_processing

Bakehouse Sales Starter Space

01_kafka_stream_processing

Python Full

Just now (6s)

5

SQL

Run all

Serverless

Schedule

Comments

Share

File Edit View Run Help

Python Full

Last edit was now

%sql

SELECT key, partition, offset, kafka_ts, ingested_at, left(value, 60) AS value_start

FROM atliq.streaming.bronze_order_events

ORDER BY kafka_ts DESC LIMIT 10;

See performance (1)

Checked for improvements

Table

key

partition

offset

kafka_ts

ingested_at

value_start

1

1790841340

3

151

2026-10-01T07:55:26.658+00...

2026-10-01T18:58:27.992+00...

("event_id": "732b4cce-f145-4ce9-b3cb-17b948df3e11", "event_

2

1790841368

5

129

2026-10-01T07:55:26.157+00...

2026-10-01T18:58:27.992+00...

("event_id": "aa7b73bf-71ba-4f48-beda-a65398d8f529", "event_

3

1790841368

5

130

2026-10-01T07:55:26.157+00...

2026-10-01T18:58:27.992+00...

("event_id": "a99e3e25-f9b5-4adc-b3c0-1a91d24e7daa", "event_

4

1790841367

4

107

2026-10-01T07:55:25.656+00...

2026-10-01T18:58:27.992+00...

("event_id": "d5e2fed6-6c9f-4423-aea7-619272068289", "event_

5

1790841331

2

180

2026-10-01T07:55:25.155+00...

2026-10-01T18:58:27.992+00...

("event_id": "da1140d7-29c3-45fe-b7d2-8db6e203c72f", "event_

6

1790841344

2

179

2026-10-01T07:55:24.654+00...

2026-10-01T18:58:27.992+00...

("event_id": "3b7b0d91-fecb-4235-978b-36602dfa3c31", "event_

7

1790841366

0

149

2026-10-01T07:55:24.152+00...

2026-10-01T18:58:27.992+00...

("event_id": "ec592a4f-53a3-435e-a7cb-f1aab073fdff", "event_

8

1790841365

0

147

2026-10-01T07:55:23.651+00...

2026-10-01T18:58:27.992+00...

("event_id": "12238ba5-6c3b-4a44-a56d-01003eef7217", "event_

9

1790841365

0

148

2026-10-01T07:55:23.651+00...

2026-10-01T18:58:27.992+00...

("event_id": "346c9d21-c944-433a-9c8a-ebc53ffeb5c4", "event_

10

1790841364

2

178

2026-10-01T07:55:23.149+00...

2026-10-01T18:58:27.992+00...

("event_id": "c03e2a2d-b5ee-47fb-b562-8cf2be7ba65e", "event_

10 rows

6.402s runtime

Refreshed at 10/2/26, 12:33 AM (Now)

This result is stored as _sqldf and can be used in other Python and SQL cells.

TASK 2 — Silver: parse, de-duplicate, handle late data

Stream FROM the Bronze table into atliq.streaming.silver_order_events :

1. Parse the JSON value with an explicit schema (event_id, event_type, event_ts, order_id, customer_id, city, product_id, quantity, order_amount, payment_method).

2. Convert event_ts to a real timestamp.

3. Add a 10-minute watermark on event_ts, then dropDuplicates(["event_id"]) — so a replayed event can never land twice.

Hint: spark.readStream.table(...), F.from_json, withWatermark.

Bronze sample: Kafka metadata, raw JSON value, and the ingestion timestamp.

4

STEP 4

Parse and De-duplicate into Silver

WHAT

Streamed from the Bronze table, parsed the JSON with an explicit schema, cast event_ts to a timestamp, dropped unparseable records, applied a 10-minute watermark and de-duplicated on event_id with event_ts, writing to atliq.streaming.silver_order_events.

WHY

An explicit schema means types are decided by design, not guessed from data. Including the watermarked event_ts in the dedup key gives the same result as event_id alone, because a replay carries the same timestamp, but lets Spark discard old dedup state instead of remembering every ID forever.

STEP 4 | Parse and De-duplicate into Silver (1 of 2)

```
57 - # TODO: Task 2 - your Silver stream here

113 +
114 + # Explicit schema: types are decided by us, not guessed from the data
115 + event_schema = T.StructType([
116 +     T.StructField("event_id", T.StringType()),
117 +     T.StructField("event_type", T.StringType()),
118 +     T.StructField("event_ts", T.StringType()),
119 +     T.StructField("order_id", T.LongType()),
120 +     T.StructField("customer_id", T.IntegerType()),
121 +     T.StructField("city", T.StringType()),
122 +     T.StructField("product_id", T.IntegerType()),
123 +     T.StructField("quantity", T.IntegerType()),
124 +     T.StructField("order_amount", T.DecimalType(12, 2)),
125 +     T.StructField("payment_method", T.StringType()),
126 + ])
127 +
128 + silver_df = (
129 +     spark.readStream.table(BRONZE)
130 +     .select(F.from_json("value", event_schema).alias("e"), "partition", "offset", "kafka_ts")
131 +     .select("e.*", "partition", "offset", "kafka_ts")
132 +     .filter(F.col("event_id").isNotNull()) # drop records that failed to parse
133 +     .withColumn("event_ts", F.to_timestamp("event_ts")) # ISO string -> timestamp (UTC)
134 +     .withWatermark("event_ts", "10 minutes") # tolerate events up to 10 min late
135 +     .dropDuplicates(["event_id", "event_ts"]) # a replayed event never lands twice
136 + )
137 +
138 + silver_q = (
139 +     silver_df.writeStream
140 +     .outputMode("append")
141 +     .option("checkpointLocation", f"{CKPT}/silver") # Silver's own checkpoint
142 +     .trigger(availableNow=True)
143 +     .toTable(SILVER)
144 + )
145 + silver_q.awaitTermination()
146 + print("Silver rows:", spark.table(SILVER).count())
147
148
149 # COMMAND -----
150 +
```

Commit 29d3134: the Silver TODO replaced by from_json, watermark, dedup and a separate checkpoint.

STEP 4 | Parse and De-duplicate into Silver (2 of 2)

databricks
Search data, notebooks, recents, and more... CTRL + P
Verify identity workspace

New

- Home
- Learn
- Workspace**
- Recents
- Catalog
- Jobs & Pipelines
- Compute
- Discover
- Marketplace

SQL

- SQL Editor
- Queries
- Dashboards
- Genie Agents
- Alerts
- Query History
- SQL Warehouses

Data Engineering

- Runs
- Data Ingestion
- Visual Data Prep

AI/ML

- Playground
- Agents
- AI Gateway
- Experiments
- Features
- Models
- Serving

Tree view: ON

Type to search

- 01_kafka_stream_processing
- Bakehouse Sales Starter Space

01_kafka_stream_processing x +

File Edit View Run Help Python Full Last edit was now

```

python --help
--select(F.from_json("value", event_schema).alias("e"), "partition", "offset", "kafka_ts")
--select("e.*", "partition", "offset", "kafka_ts")
--filter(F.col("event_id").isNull()) # drop records that failed to parse
--withColumn("event_ts", F.to_timestamp("event_ts")) # ISO string -> timestamp (UTC)
--withWatermark("event_ts", "10 minutes") # tolerate events up to 10 min late
--dropDuplicates(["event_id", "event_ts"]) # a replayed event never lands twice

silver_q = (
    silver_df.writeStream
        .outputMode("append")
        .option("checkpointLocation", f"{CKPT}/silver") # Silver's own checkpoint
        .trigger(availableNow=True)
        .toTable(SILVER)
)

silver_q.awaitTermination()
print(f"Silver rows: ", spark.table(SILVER).count())
          
```

> See performance (2)

> silver_df: pyspark.sql.connect.dataframe.DataFrame = [event_id: string, event_type: string ... 11 more fields]

Silver rows: 879

Just now (2s) 8 SQL

```
%sql
SELECT event_type, COUNT(*) AS events
FROM atliq.streaming.silver_order_events GROUP BY event_type ORDER BY events DESC;
```

> See performance (1)

event_type	events
order_placed	342
payment_received	281
order_shipped	209
order_cancelled	47

4 rows 2.320s runtime Refreshed at 10/2/26, 12:41 AM (Now)

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

First Silver run: 879 rows across order_placed, payment_received, order_shipped and order_cancelled.



STEP 5

Run Incrementally

WHAT

Sent a second producer burst of 343 events, then re-ran the Bronze and Silver streams. Both tables grew to 1,222 rows.

WHY

879 plus 343 is exactly 1,222. Each stream's checkpoint recorded where it stopped, so the second run read only the new events: no full reload, no re-reading old offsets, nothing counted twice. This is the incremental load Phase 1 built with MERGE, here done by the stream itself.

STEP 5 | Run Incrementally (1 of 2)

```
1  """
2  AtliQ Commerce – Phase 2 (LEARNER STARTER)
3  Live Order Event Producer – stream simulated order events to Kafka.
4
5  YOUR JOB: complete the TODOs. The event shapes and helper pieces are given so
6  you focus on the Kafka parts: configuring the producer and publishing events.
7
8  Prereqs:
9      pip install confluent-kafka python-dotenv
10     .env (next to this file – copy .env.example):
11     KAFKA_BOOTSTRAP=pkc-xxxxx.region.provider.confluent.cloud:9092
12     KAFKA_API_KEY=...
13     KAFKA_API_SECRET=...
```

```
OK key=1790882325 -> atliq.orders.events [partition 1] @ offset 203
OK key=1790882325 -> atliq.orders.events [partition 1] @ offset 204
OK key=1790882326 -> atliq.orders.events [partition 3] @ offset 197
OK key=1790882326 -> atliq.orders.events [partition 3] @ offset 198
OK key=1790882327 -> atliq.orders.events [partition 3] @ offset 199
OK key=1790882327 -> atliq.orders.events [partition 3] @ offset 200
OK key=1790882322 -> atliq.orders.events [partition 4] @ offset 182
OK key=1790882301 -> atliq.orders.events [partition 4] @ offset 183
OK key=1790882302 -> atliq.orders.events [partition 2] @ offset 223
OK key=1790882319 -> atliq.orders.events [partition 5] @ offset 190
OK key=1790882328 -> atliq.orders.events [partition 4] @ offset 184
OK key=1790882328 -> atliq.orders.events [partition 4] @ offset 185
OK key=1790882329 -> atliq.orders.events [partition 2] @ offset 224
OK key=1790882329 -> atliq.orders.events [partition 2] @ offset 225
OK key=1790882328 -> atliq.orders.events [partition 4] @ offset 186
OK key=1790882330 -> atliq.orders.events [partition 5] @ offset 191
OK key=1790882331 -> atliq.orders.events [partition 1] @ offset 205
OK key=1790882331 -> atliq.orders.events [partition 1] @ offset 206
OK key=1790882332 -> atliq.orders.events [partition 1] @ offset 207
OK key=1790882332 -> atliq.orders.events [partition 1] @ offset 208
OK key=1790882324 -> atliq.orders.events [partition 3] @ offset 201
OK key=1790882314 -> atliq.orders.events [partition 2] @ offset 226
OK key=1790882333 -> atliq.orders.events [partition 1] @ offset 209
OK key=1790882333 -> atliq.orders.events [partition 1] @ offset 210
OK key=1790882325 -> atliq.orders.events [partition 1] @ offset 211
OK key=1790882334 -> atliq.orders.events [partition 0] @ offset 190
OK key=1790882334 -> atliq.orders.events [partition 0] @ offset 191
OK key=1790882305 -> atliq.orders.events [partition 5] @ offset 192
OK key=1790882335 -> atliq.orders.events [partition 0] @ offset 192
OK key=1790882335 -> atliq.orders.events [partition 0] @ offset 193
OK key=1790882336 -> atliq.orders.events [partition 4] @ offset 187
OK key=1790882337 -> atliq.orders.events [partition 0] @ offset 194
OK key=1790882338 -> atliq.orders.events [partition 5] @ offset 193
OK key=1790882338 -> atliq.orders.events [partition 5] @ offset 194
OK key=1790882338 -> atliq.orders.events [partition 5] @ offset 195
OK key=1790882339 -> atliq.orders.events [partition 3] @ offset 202
OK key=1790882313 -> atliq.orders.events [partition 1] @ offset 212
Done. 343 events sent, 0 undelivered.
PS F:\Github Cloned Repos\atliq-commerce-capstone\phase2-streaming\producer>
```

Second producer run: Done. 343 events sent, 0 undelivered.

STEP 5 | Run Incrementally (2 of 2)

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

01_kafka_stream_processing

File Edit View Run Help Python Full Last edit was 13 minutes ago

1 minute ago (34s)

7

```
spark.readStream.table(BRONZE)
.select(F.from_json("value", event_schema).alias("e"), "partition", "offset", "kafka_ts")
.select("e.*", "partition", "offset", "kafka_ts")
.filter(F.col("event_id").isNull())           # drop records that failed to parse
.withColumn("event_ts", F.to_timestamp("event_ts")) # ISO string -> timestamp (UTC)
.withWatermark("event_ts", "10 minutes")      # tolerate events up to 10 min late
.dropDuplicates(["event_id", "event_ts"])      # a replayed event never lands twice

)

silver_q = (
    silver_df.writeStream
        .outputMode("append")
        .option("checkpointLocation", f"{CKPT}/silver") # Silver's own checkpoint
        .trigger(availableNow=True)
        .toTable(SILVER)
)
silver_q.awaitTermination()
print("Silver rows:", spark.table(SILVER).count())
```

[See performance \(2\)](#)

silver_df: pyspark.sql.connect.dataframe.DataFrame = [event_id: string, event_type: string ... 11 more fields]

Silver rows: 1222

Just now (2s)

8

SQL

See performance (1)

%sql

SELECT event_type, COUNT(*) AS events

FROM atliq_streaming.silver_order_events GROUP BY event_type ORDER BY events DESC;

Table

	event_type	events
1	order_placed	470
2	payment_received	385
3	order_shipped	295
4	order_cancelled	72

4 rows

2.440s runtime

Refreshed at 10/2/26, 12:55 AM (Now)

This result is stored as _sqldf and can be used in other Python and SQL cells.

Silver after run 2: 1,222 rows, every event type higher than run 1.



STEP 6

Prove There Are No Duplicates

WHAT

Compared the total Silver row count with the number of distinct event_ids.

WHY

1,222 rows and 1,222 distinct events. event_id plays the role updated_at played in the batch lane: the key that makes a replayed or redelivered event harmless.

STEP 6 | Prove There Are No Duplicates

data

bricks

Fast. Collaborative.

+ New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Home

Tree view: ON

← .../ ashish@alp...

Type to search

01_kafka_stream_processing

Bakehouse Sales Starter Space

01_kafka_stream_processing

File Edit View Run Help Python Full Last edit was now

Run all Serverless Schedule Comments Share

DEDUP PROOF

1 minute ago (2s) 10

%sql
SELECT COUNT(*) AS silver_rows, COUNT(DISTINCT event_id) AS distinct_events
FROM atliq.streaming.silver_order_events;

See performance (1) Optimize

Table +

	silver_rows	distinct_events
1	1222	1222

Download 1 row 1.882s runtime Refreshed at 10/2/26, 12:58 AM (1m ago)

This result is stored as _sqlidf and can be used in other Python and SQL cells.

TASK 3 — Gold: the live revenue ticker

From the Silver stream, keep only payment_received events and aggregate into 5-minute tumbling windows: orders_paid = count, revenue = sum of order_amount. Append closed windows to atliq.streaming.gold_revenue_5min .

Hint: F.window("event_ts", "5 minutes") — and think about WHY a window only appears after the watermark passes its end (you will explain this in your write-up).

12

TODO: Task 2 – your Silver stream here

Verify (given)

Dedup proof: silver_rows 1,222 equals distinct_events 1,222.



STEP 7

Trace One Order End to End

WHAT

Picked order 1790841036 from the Task 1 producer output and traced it through Silver: order_placed, payment_received, then order_cancelled.

WHY

All three events sit on partition 2 with rising offsets 2, 3 and 8, in lifecycle order. That is the payoff of keying events by order_id: Kafka guarantees order only within a partition, and equal keys always land in the same one.

STEP 7 | Trace One Order End to End

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Home

Tree view: ON

ashish@alp...

Type to search

01_kafka_stream_processing

Bakehouse Sales Starter Space

01_kafka_stream_processing

File Edit View Run Help Python Full Last edit was now

Order Trace

Just now (2s)

12

SQL

```
%sql
SELECT order_id, event_type, event_ts, partition, offset
FROM atliq.streaming.silver_order_events
WHERE order_id = 1790841036
ORDER BY offset;
```

See performance (1)

Checked for improvements

Table

	order_id	event_type	event_ts	partition	offset
1	1790841036	order_placed	2026-10-01T07:50:34.136+00:...	2	2
2	1790841036	payment_received	2026-10-01T07:50:34.136+00:...	2	3
3	1790841036	order_cancelled	2026-10-01T07:50:38.651+00:...	2	8

3 rows

1.685s runtime

Refreshed at 10/2/26, 1:01 AM (Now)

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

TASK 3 — Gold: the live revenue ticker

From the Silver stream, keep only `payment_received` events and aggregate into **5-minute tumbling windows**: `orders_paid = count`, `revenue = sum of order_amount`. Append closed windows to `atliq.streaming.gold_revenue_5min`.

Hint: `F.window("event_ts", "5 minutes")` — and think about WHY a window only appears after the watermark passes its end (you will explain this in your write-up).

14

TODO: Task 2 – your Silver stream here

Verify (given)

Order 1790841036: placed, paid, cancelled; one partition, offsets in sequence.



STEP 8

Governed Tables and Checkpoints

WHAT

Both tables are managed Delta tables in Unity Catalog under `atliq.streaming`, next to the checkpoints volume that holds a separate checkpoint folder for each stream.

WHY

Separate checkpoints per stream are what make restarts safe: each stream resumes from its own bookmark with no loss and no duplicates. Unity Catalog gives the tables an owner, lineage back to the notebook, and one governed place to find them.

STEP 8 | Governed Tables and Checkpoints (1 of 2)

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Catalog

Serverless Starter Warehouse Serverless 2XS

Catalog Explorer

atliq streaming

bronze_order_events

Overview Sample Data Details Permissions Policies History Lineage Insights Quality

Description

AI generate Add

Filter columns...

AI generate

Column	Type	Popularity	Comment	Tags	Column masking
key	string				
value	string				
topic	string	----			
partition	int				
offset	bigint				
kafka_ts	timestamp				
ingested_at	timestamp				

My organization

workspace

default

information_schema

system

atliq

default

information_schema

streaming

Tables (2)

bronze_order_events

silver_order_events

Volumes (1)

checkpoints

Shares received

samples

Verify identity

workspace

Share

Create

About this table

Owner ashish@alphainterbiz.com

Type Managed

Data source Delta

Popularity |||

Quality

Not enabled

Data Quality Monitoring is not enabled for this table

Enable

Policies

New policy

Tags

Add tags

Row filter

Add filter

Insights

Top users

ashish@alphainterbiz.com

No tables joined in past 30 days

Related assets

01_kafka_stream_processing

Insights

atliq.streaming: bronze_order_events schema, both tables, and the checkpoints volume.

STEP 8 | Governed Tables and Checkpoints (2 of 2)

databricks

Free Edition

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Serverless Starter Warehouse

Serverless

2XS

Catalog

Type to search

My organization

workspace

default

information_schema

system

atliq

default

information_schema

streaming

Tables (2)

bronze_order_events

silver_order_events

Volumes (1)

checkpoints

Shares received

samples

Catalog Explorer > atliq > streaming >

silver_order_events

Overview Sample Data Details Permissions Policies History Lineage Insights Quality

AI Suggested Description

The table captures events related to customer orders. It includes details such as the type of event, timestamps, order and customer IDs, as well as information about the products ordered and their quantities. This data can be useful for tracking order lifecycle events, analyzing purchasing trends, and understanding customer behavior.

Accept Edit

Filter columns...

AI generate

Column	Type	Popularity	Comment	Tags	Column masking
event_id	string				
event_type	string				
event_ts	timestamp				
order_id	bigint				
customer_id	int				
city	string				
product_id	int				
quantity	int				
order_amount	decimal(12,2)				
payment_method	string				
partition	int				
offset	bigint				
kafka_ts	timestamp				

About this table

Owner ashish@alphainterbiz.com

Type Managed

Data source Delta

Popularity

Quality

Not enabled

Data Quality Monitoring is not enabled for this table

Enable

Policies

New policy

Tags

Add tags

Row filter

Add filter

Insights

Top users

ashish@alphainterbiz.com

No tables joined in past 30 days

Related assets

01_kafka_stream_processing

Insights

silver_order_events: typed columns, including event_ts as timestamp and order_amount as decimal(12,2).

Task 2 Complete

What this task delivered

Raw events landed

Kafka to Bronze over SASL_SSL with credentials from a secret scope; every producer event landed, with its Kafka metadata kept.

Clean, typed Silver

Explicit schema, real timestamps, a 10-minute watermark and de-duplication on event_id, with bounded streaming state.

Incremental and exact

Checkpoints made the second run read only new events: 879 plus 343 became 1,222 with zero duplicates.

Reconciliation

879

rows after run 1

+343

new events, run 2

1,222

rows, Bronze and Silver

0

duplicates

Engineering decisions: Secret scope instead of credentials in the committed notebook; availableNow trigger (serverless) with awaitTermination so Silver reads a finished Bronze; separate checkpoint per stream in a UC volume; dedup on (event_id, event_ts) under the watermark to bound state; startingOffsets earliest for the first run only.